

TP Python — Mon herbier numérique

1. Contexte

Vous souhaitez construire un **herbier numérique** permettant de mémoriser et de réviser les principales caractéristiques de plantes étudiées en botanique.

L'application devra permettre d'enregistrer des plantes, de rechercher et filtrer les plantes de l'herbier, de sauvegarder les données au format JSON et de proposer un **quiz automatique sur les familles botaniques**.

Ce projet a pour objectif de mobiliser plusieurs notions de programmation :

- programmation orientée objet ;
 - classes, attributs et méthodes ;
 - listes et dictionnaires ;
 - recherche et filtrage ;
 - gestion des chaînes de caractères ;
 - validation et nettoyage des données ;
 - lecture et écriture de fichiers ;
 - sérialisation JSON ;
 - génération aléatoire de questions ;
 - gestion des erreurs ;
 - conception d'un programme structuré en fonctions.
-

2. Objectifs pédagogiques

À la fin du projet, l'étudiant devra être capable de :

1. définir une classe Python adaptée à un problème ;
 2. créer et manipuler des objets ;
 3. écrire des méthodes de recherche et de filtrage ;
 4. traiter des données imparfaites ;
 5. sérialiser et désérialiser des objets avec JSON ;
 6. utiliser le module `random` ;
 7. construire un quiz automatique ;
 8. structurer un programme en plusieurs fonctions ;
 9. produire un programme robuste face aux données incorrectes ;
 10. expliquer les choix effectués et la complexité des principales opérations.
-

3. Cahier des charges

L'application devra gérer un ensemble de plantes.

Chaque plante possède au minimum les informations suivantes :

- **nom vernaculaire ;**
- **nom scientifique ;**
- **famille botanique ;**
- **cycle ;**
- **besoins ;**
- **photo.**

On utilisera la classe :

Plante

avec au minimum les attributs :

```
nom
nom_scientifique
famille
cycle
besoins
photo
```

4. Partie A — Classe `Plante`

Créer une classe `Plante`.

Exemple de construction :

```
class Plante:

    def __init__(
        self,
        nom,
        nom_scientifique,
        famille,
        cycle,
        besoins,
        photo
    ):
        ...
```

4.1 Représentation textuelle

Implémenter la méthode :

```
__str__()
```

afin que l'instruction :

```
print(plante)
```

produise une information lisible, par exemple :

```
Tomate (Solanum lycopersicum) — Solanaceae  
Cycle : annuel  
Besoins : soleil, arrosage régulier
```

Travail demandé

Créer au moins trois objets `Plante` et les afficher.

5. Partie B — Gestion de l'herbier

Créer une classe :

```
Herbier
```

qui contient une collection de plantes.

Elle devra proposer au minimum les méthodes suivantes :

```
ajouter_plante(plante)  
supprimer_plante(...)  
rechercher(...)  
filtrer_par_famille(...)  
filtrer_par_cycle(...)  
filtrer_par_besoin(...)  
afficher(...)
```

Vous êtes libre de choisir la structure interne :

```
list
```

ou

```
dict
```

ou une combinaison des deux.

6. Partie C — Recherche

L'utilisateur doit pouvoir rechercher une plante à partir de son nom.

La recherche devra être :

- insensible à la casse ;
- tolérante aux espaces inutiles ;

- capable de rechercher une partie du nom.

Exemple :

Recherche : tom

peut retourner :

Tomate
Tomatillo

Question

Quelle est la complexité de votre méthode de recherche si les plantes sont stockées dans une liste de taille n ?

7. Partie D — Filtrage

L'application devra proposer des filtres.

Filtrer par famille

Exemple :

```
herbier.filtrer_par_famille("Solanaceae")
```

Filtrer par cycle

Valeurs possibles :

annuel
bisannuel
vivace

Filtrer par besoin

Par exemple :

soleil
mi-ombre
ombre
arrosage_faible
arrosage_moyen
arrosage_frequent

Un utilisateur doit pouvoir demander :

plantes de la famille Fabaceae

ou :

plantes vivaces ayant besoin d'ombre

8. Partie E — Jeu de données volontairement dégradé

Le fichier `plantes_degradees.json` fourni avec le sujet contient les données initiales de l'herbier.

Attention : ce fichier est volontairement imparfait.

Il contient différents problèmes que le programme doit être capable de détecter et, lorsque cela est possible, de corriger.

Créer un fichier `plantes_degradees.json` contenant les données suivantes :

```
[
  {
    "nom": "Tomate",
    "nom_scientifique": "Solanum lycopersicum",
    "famille": "Solanaceae",
    "cycle": "annuel",
    "besoins": ["soleil", "arrosage regulier"],
    "photo": "tomate.jpg"
  },
  {
    "nom": "tomate ",
    "nom_scientifique": "Solanum lycopersicum",
    "famille": "Solanacées",
    "cycle": "Annuel",
    "besoins": ["soleil", "arrosage régulier"],
    "photo": "tomate.jpg"
  },
  {
    "nom": "Tournesol",
    "nom_scientifique": "Helianthus annuus",
    "famille": "Asteraceae",
    "cycle": "annuel",
    "besoins": ["soleil"],
    "photo": "tournesol.jpg"
  },
  {
    "nom": "Haricot",
    "nom_scientifique": "Phaseolus vulgaris",
    "famille": "Fabaceae",
    "cycle": "annuel",
    "besoins": ["soleil", "arrosage moyen"],
    "photo": "haricot.jpg"
  },
]
```

```

{
  "nom": "Pois",
  "nom_scientifique": "Pisum sativum",
  "famille": "FABACEAE",
  "cycle": "annuel",
  "besoins": ["soleil", "arrosage moyen"],
  "photo": "pois.jpg"
},

{
  "nom": "Lavande",
  "nom_scientifique": "Lavandula angustifolia",
  "famille": "Lamiaceae",
  "cycle": "vivace",
  "besoins": ["soleil", "arrosage_faible"],
  "photo": "lavande.jpg"
},

{
  "nom": "Menthe",
  "nom_scientifique": "Mentha sp.",
  "famille": "Lamiaceae",
  "cycle": "VIVACE",
  "besoins": ["mi-ombre", "arrosage fréquent"],
  "photo": null
},

{
  "nom": "Chêne",
  "nom_scientifique": "Quercus robur",
  "famille": "Fagaceae",
  "cycle": "vivace",
  "besoins": ["soleil"],
  "photo": "chene.jpg"
},

{
  "nom": "Rose",
  "nom_scientifique": "Rosa gallica",
  "famille": "Rosaceae",
  "cycle": "Vivace",
  "besoins": ["soleil", "arrosage_moyen"],
  "photo": "rose.jpg"
},

{
  "nom": "Pissenlit",
  "nom_scientifique": "Taraxacum officinale",
  "famille": "asteraceae",
  "cycle": "vivace",
  "besoins": ["soleil"],
  "photo": "pissenlit.jpg"
},

{
  "nom": "",
  "nom_scientifique": "Ocimum basilicum",
  "famille": "Lamiaceae",
  "cycle": "annuel",
  "besoins": ["soleil", "arrosage fréquent"],
  "photo": "basilic.jpg"
}

```

```

    },
    {
      "nom": "Romarin",
      "nom_scientifique": "Salvia rosmarinus",
      "famille": "Lamiaceae",
      "cycle": "vivace",
      "besoins": ["soleil", "arrosage faible"],
      "photo": "romarin.jpg"
    },
    {
      "nom": "Carotte",
      "nom_scientifique": "Daucus carota",
      "famille": "Apiaceae",
      "cycle": "bisannuel",
      "besoins": ["soleil", "arrosage moyen"],
      "photo": "carotte.jpg"
    },
    {
      "nom": "Carotte",
      "nom_scientifique": "Daucus carota",
      "famille": "Apiaceae",
      "cycle": "bisannuel",
      "besoins": ["soleil", "arrosage moyen"],
      "photo": "carotte.jpg"
    },
    {
      "nom": "Ortie",
      "nom_scientifique": "Urtica dioica",
      "famille": "Urticaceae",
      "cycle": "vivace",
      "besoins": ["mi-ombre"],
      "photo": "ortie.jpg"
    },
    {
      "nom": "Basilic",
      "nom_scientifique": "Ocimum basilicum",
      "famille": "Lamiaceae",
      "cycle": "annuel",
      "besoins": ["soleil", "arrosage frequent"],
      "photo": "basilic.jpg"
    }
  ]

```

9. Travail de nettoyage des données

Le programme doit être capable d'identifier au minimum les problèmes suivants :

Problème 1 — Espaces

"tomate "

doit être considéré comme :

"Tomate"

Problème 2 — Casse

Les valeurs :

"Solanaceae"
"Solanacées"
"Solanaceae"

doivent faire l'objet d'une réflexion sur la normalisation choisie.

Problème 3 — Cycle

Les valeurs :

annuel
Annuel
VIVACE
Vivace

doivent être normalisées.

Problème 4 — Accents et écriture des besoins

On trouve :

arrosage regulier
arrosage régulier
arrosage_faible
arrosage faible
arrosage fréquent
arrosage frequent

Le programme devra définir une convention interne.

Problème 5 — Valeur manquante

Le basilic possède un nom vide :

"nom": ""

L'entrée doit être signalée.

Problème 6 — Photo absente

La menthe possède :

"photo": null

Ce cas doit être géré sans faire planter le programme.

Problème 7 — Doublons

La carotte apparaît deux fois.

Le programme doit être capable de détecter ce doublon.

Problème 8 — Variantes linguistiques

Le jeu de données utilise parfois :

Solanaceae
Solanacées

Le programme devra définir une convention et documenter son choix.

10. Fonction de nettoyage

Implémenter une fonction :

```
nettoyer_plante(donnees)
```

Elle devra :

1. supprimer les espaces superflus ;
2. normaliser la casse ;
3. normaliser les valeurs de `cycle` ;
4. normaliser les besoins ;
5. vérifier les champs obligatoires ;
6. signaler les données manquantes ;
7. retourner une structure cohérente.

Exemple attendu :

```
{  
  "nom": "Tomate",  
  "nom_scientifique": "Solanum lycopersicum",  
  "famille": "Solanaceae",  
  "cycle": "annuel",  
  "besoins": ["soleil", "arrosage_regulier"],  
  "photo": "tomate.jpg"  
}
```

11. Validation des données

Créer une fonction :

```
valider_plante(plante)
```

qui vérifie par exemple :

- le nom n'est pas vide ;
- le nom scientifique n'est pas vide ;
- la famille est renseignée ;
- le cycle appartient à une liste autorisée ;
- les besoins sont une liste ;
- la photo peut être absente.

La fonction pourra retourner :

`True`

ou :

`False`

ou, mieux encore, un rapport d'erreurs.

Exemple :

`Basilic`
`Erreur : nom vernaculaire manquant`

12. Détection des doublons

Deux entrées peuvent être considérées comme identiques si elles possèdent le même :

`nom scientifique`

Travail demandé

Implémenter :

`detecter_doublons(plantes)`

La fonction doit retourner les doublons détectés.

Question algorithmique

Quelle est la complexité d'une recherche naïve de doublons ?

Comment pourrait-on améliorer cette recherche à l'aide d'un dictionnaire Python ?

13. Sérialisation JSON

Ajouter à la classe `Herbier` les méthodes :

```
sauvegarder_json(nom_fichier)
charger_json(nom_fichier)
```

Exemple :

```
herbier.sauvegarder_json("mon_herbier.json")
```

Puis :

```
herbier2 = Herbier()
herbier2.charger_json("mon_herbier.json")
```

Après rechargement, les données doivent être équivalentes.

Question

Pourquoi la sérialisation est-elle utile ?

Donner au moins deux applications possibles.

14. Menu de l'application

Construire un menu permettant à l'utilisateur d'effectuer les opérations suivantes :

```
=====
MON HERBIER NUMÉRIQUE
=====
```

1. Afficher les plantes
2. Rechercher une plante
3. Filtrer par famille
4. Filtrer par cycle
5. Filtrer par besoin
6. Ajouter une plante
7. Supprimer une plante
8. Sauvegarder l'herbier
9. Charger l'herbier
10. Lancer le quiz
0. Quitter

Le programme doit rester actif tant que l'utilisateur ne choisit pas 0.

15. Quiz automatique

L'application doit proposer un quiz permettant de réviser les familles botaniques.

Le programme choisit aléatoirement une plante et pose une question.

Exemple :

Quelle est la famille botanique de la tomate ?

- A. Rosaceae
- B. Solanaceae
- C. Fabaceae
- D. Lamiaceae

L'utilisateur saisit :

B

Le programme répond :

Bonne réponse !

ou :

Mauvaise réponse.

La bonne réponse était : Solanaceae

16. Génération des choix

Pour chaque question :

- une bonne réponse ;
- trois mauvaises réponses.

Les trois mauvaises réponses doivent être choisies parmi les familles présentes dans l'herbier.

Les réponses doivent être mélangées aléatoirement.

Utiliser :

```
import random
```

et notamment :

```
random.choice(...)
random.sample(...)
random.shuffle(...)
```

17. Score du quiz

Le quiz comporte par exemple **5 questions**.

À la fin, afficher :

```
=====
                        RESULTAT
=====
```

Score : 4 / 5

80 %

Puis une appréciation :

```
5/5  → Excellent !
4/5  → Très bien !
3/5  → Bien, mais encore un peu de révision.
0-2/5 → Revoir les familles botaniques.
```

18. Éviter les répétitions

Une même plante ne doit pas être proposée deux fois au cours du même quiz.

À vous de jouer

Mettre en œuvre une stratégie permettant de sélectionner 5 plantes différentes.

19. Mode révision

Ajouter un mode permettant de filtrer le quiz.

Exemple :

Choisir une famille :

1. Fabaceae
2. Lamiaceae
3. Rosaceae
4. Asteraceae
5. Toutes

Si l'utilisateur choisit `Lamiaceae`, toutes les questions du quiz doivent porter sur cette famille.

20. Gestion des erreurs

Le programme doit rester fonctionnel lorsqu'un utilisateur saisit :

abc

alors qu'un nombre est attendu.

Il doit également gérer :

- un fichier JSON inexistant ;
- un fichier JSON mal formé ;
- une plante inconnue ;
- une recherche sans résultat ;
- une liste de familles trop courte pour générer quatre propositions.

Utiliser notamment :

```
try:
    ...
except:
    ...
```

mais en privilégiant des exceptions précises :

```
except FileNotFoundError:
except json.JSONDecodeError:
```

21. Architecture attendue

Il est fortement recommandé d'organiser le programme avec une structure proche de :

```
herbier/
├── plante.py
├── herbier.py
├── quiz.py
├── donnees.py
└── main.py
```

Une version simplifiée dans un seul notebook est également acceptée.

22. Extensions facultatives

Les étudiants souhaitant aller plus loin peuvent ajouter :

Extension 1 — Recherche avancée

Recherche par plusieurs critères :

```
famille = Lamiaceae
ET
```

cycle = vivace

Extension 2 — Statistiques

Afficher :

- nombre total de plantes ;
- nombre de familles ;
- famille la plus représentée ;
- nombre de plantes par cycle.

Extension 3 — Score cumulatif

Conserver les scores de plusieurs quiz.

Extension 4 — Difficulté adaptative

Proposer davantage de questions sur les familles mal maîtrisées.

Extension 5 — Interface graphique

Créer une interface avec `tkinter`.

Extension 6 — Photos

Afficher les photographies dans une interface graphique.

23. Questions d'analyse algorithmique

Répondre aux questions suivantes dans le rapport.

Question 1

Si l'herbier contient n plantes, quelle est la complexité d'une recherche linéaire par nom ?

Question 2

Comment utiliser un dictionnaire pour accélérer la recherche par nom scientifique ?

Question 3

Quelle serait la complexité d'une recherche dans ce dictionnaire ?

Question 4

Pourquoi faut-il nettoyer les données avant d'effectuer les recherches ?

Question 5

Quelle est la différence entre :

donnée brute
donnée nettoyée
objet Python
représentation JSON

Question 6

Pourquoi un quiz utilisant `random.choice()` peut-il poser plusieurs fois la même question ?

Comment éviter ce problème ?

24. Jeu de tests minimal

Le programme devra être testé sur au moins les cas suivants.

Test	Résultat attendu
Recherche <code>tom</code>	Tomate
Recherche <code>TOMATE</code>	Tomate
Famille <code>Fabaceae</code>	Haricot, Pois
Cycle <code>vivace</code>	plusieurs plantes
Besoin <code>soleil</code>	plusieurs plantes
Plante inexistante	message explicite
Nom vide	erreur détectée
Photo <code>null</code>	pas de plantage
Carotte en double	doublon détecté
Chargement JSON	données correctement restaurées
Quiz de 5 questions	score correct

25. Livrables

Le projet devra contenir :

Livrable 1 — Programme

Le code source Python.

Livrable 2 — Jeu de données

Le fichier :

`plantes_degradees.json`

et, après nettoyage :

`mon_herbier.json`

Livrable 3 — Documentation

Une courte documentation expliquant :

- les classes ;
- les structures de données ;
- les principales fonctions ;
- la stratégie de nettoyage ;
- la gestion des erreurs.

Livrable 4 — Tests

Une liste de tests réalisés et leur résultat.

Livrable 5 — Rapport

Un document de **2 à 4 pages** présentant :

- le modèle objet ;
- les choix de structures de données ;
- les algorithmes de recherche et de filtrage ;
- la sérialisation ;
- le fonctionnement du quiz ;
- l'analyse de complexité ;
- les difficultés rencontrées.

26. Grille d'évaluation — 20 points

Critère	Points
Classe <code>Plante</code> et programmation orientée objet	3 pts
Classe <code>Herbier</code> et gestion de la collection	2 pts
Recherche et filtrage	3 pts
Nettoyage et validation des données dégradées	3 pts
Sérialisation / désérialisation JSON	2 pts
Quiz automatique et génération aléatoire	3 pts
Gestion des erreurs et robustesse	1,5 pt
Tests et qualité du code	1 pt

Critère	Points
Analyse de complexité et documentation	1,5 pt
Total	20 pts

27. Critères de qualité

Un programme ne sera pas considéré comme complètement fonctionnel s'il :

- plante lorsqu'une photo est absente ;
- distingue "Tomate" et "tomate " comme deux plantes différentes sans justification ;
- ne détecte pas les doublons ;
- ne peut pas recharger les données sauvegardées ;
- génère systématiquement la même question ;
- ne gère pas les saisies incorrectes.

On évaluera également :

- la lisibilité du code ;
 - le découpage en fonctions ;
 - la pertinence des noms de variables ;
 - les commentaires utiles ;
 - la réutilisabilité du code.
-

28. Compétences évaluées

Ce projet permet notamment d'évaluer les compétences suivantes :

Programmation Python

- variables ;
- conditions ;
- boucles ;
- fonctions ;
- exceptions ;
- modules.

Structures de données

- listes ;
- dictionnaires ;
- ensembles ;
- parcours ;
- recherche.

Programmation orientée objet

- classes ;
- attributs ;
- méthodes ;
- encapsulation ;
- interaction entre objets.

Algorithmique

- recherche ;
- filtrage ;
- détection de doublons ;
- génération aléatoire ;
- analyse de complexité.

Gestion des données

- fichiers ;
- JSON ;
- nettoyage ;
- validation ;
- données manquantes.

29. Résultat attendu

À la fin du projet, l'utilisateur doit pouvoir lancer :

```
python main.py
```

et obtenir une application permettant de :

```
MON HERBIER NUMÉRIQUE
```

```
25 plantes enregistrées  
12 familles botaniques
```

1. Explorer l'herbier
2. Rechercher
3. Filtrer
4. Ajouter
5. Supprimer
6. Sauvegarder
7. Charger
8. Quiz botanique
0. Quitter

Le projet doit constituer une petite application cohérente et robuste, et non une simple collection de fonctions indépendantes.